AudioReach™ Signal Processing Framework (SPF)

Technical Overview

80-VN500-16 Rev. AA

July 30, 2020

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:49:55 GMT nijiasheng@longcheer.com

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

Restricted Distribution: Not to be distributed to anyone who is not an employee of either Qualcomm Technologies, Inc. or its affiliated companies without the express approval of Qualcomm Configuration Management. Distribution to anyone who is not an employee of either Qualcomm Incorporated or its affiliated companies is subject to applicable confidentiality agreements.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

All Qualcomm products mentioned herein are products of Qualcomm Technologies, Inc. and/or its subsidiaries.

Qualcomm is a trademark of Qualcomm Incorporated, registered in the United States and other countries. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer (“export”) laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

© 2020 Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision history

Revision	Date	Description
AA	July 2020	Initial release

Note: There is no Rev. I, O, Q, S, X, or Z per Mil. standards.

Qualcomm
Confidential - May Contain Trade Secrets
2022-08-10 09:49:55 GMT
nijiasheng@longcheer.com

Contents

1 Introduction 6

1.1 Purpose 6
1.2 Scope 6
1.3 Technical assistance 6

2 Overview 7

2.1 High-level architecture 7
2.2 Possible use cases 9
2.2.1 Audio capture and recording 9
2.2.2 Audio renderer and playback 9
2.2.3 Voice over IP (VoIP) 10
2.2.4 Circuit Switched (CS) voice call 10
2.2.5 Audio transcoding 11
2.2.6 Audio loopback 11
2.2.7 Audio detection 12

3 Framework requirements 13

3.1 General 13
3.2 Client interfaces 13
3.3 Processing chains, topologies, and graphs 14
3.4 Media formats 14
3.5 Scheduling methods 15
3.6 Resource management 15
3.7 Debugging 15

4 SPF components 16

4.1 High-level architecture of SPF 16
4.2 Functional blocks 17
4.2.1 Generic Packet Router (GPR) 17
4.2.2 Modules 19
4.2.3 Links and connections 20
4.2.4 Graph and subgraph 21
4.2.5 Containers 21
4.2.6 Audio Processing Manager (APM) 26
4.2.7 Voice Call Proxy Manager (VCPM) 30
4.2.8 Audio Module Data Base (AMDB) 31
4.2.9 Proxy Resource Manager (PRM) 33
4.2.10 Integrated Resource Monitor (IRM) 34
4.2.11 Platform and OS Abstraction Layer 35

Qualcomm
Confidential - May Contain Trade Secrets
2022-02-10 09:49:55 GMT
jijiazheng@longchear.com

4.3 Calibration and configuration 36
4.3.1 Regular and non-shared calibration 36
4.3.2 Persistent and shared calibration 37
4.3.3 Shared-persistent or global-shared calibration 37
4.4 Multi-instance and multi-core 38
4.4.1 Multi-Instance configuration 38
4.4.2 Multi-core configuration 39

5 Customizations 41
5.1 Custom module 41
5.2 Custom container 42

6 Use cases 43
6.1 Playback 43
6.2 Capture 43
6.3 Use cases with source and sink modules 44
6.4 Hands Free Profile (HFP) 44
6.5 Voice activation 44
6.6 Voice activation and playback concurrency 45
6.7 Voice call concurrencies 45

A References 46
A.1 Related documents 46
A.2 Acronyms and terms 46

Qualcomm
Confidential – May Contain Trade Secrets
2022-08-10 09:49:55 GMT
nijiasheng@longcheer.com

Figures

Figure 2-1 High-level audio software architecture 8
Figure 4-1 SPF – high level architecture 16
Figure 4-2 GPR – header fields 17
Figure 4-3 In-band and out-of-band messaging methods 18
Figure 4-4 CAPI-wrapped modules with control or data links 20
Figure 4-5 Containers 22
Figure 4-6 Example dataflow across containers 25
Figure 4-7 APM 27
Figure 4-8 APM state machine 28
Figure 4-9 Voice Call Proxy Manager 30
Figure 4-10 Audio Module Data Base 31
Figure 4-11 Proxy Resource Manager 33
Figure 4-12 Integrated Resource Monitor 34
Figure 4-13 Calibration modes 36
Figure 4-14 SPF in multi-core and multi-instance configuration 38
Figure 4-15 Playback with off-load modules – Use case designer's view 39
Figure 4-16 Playback with off-load modules – SPF implementation view 40
Figure 5-1 Custom module workflow 41

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:49:55 GMT nijiasheng@longcheer.com

1 Introduction

1.1 Purpose

This document details the high-level software requirements and design details of the Signal Processing Framework (SPF) used in the context of the AudioReach™ architecture.

1.2 Scope

This document focuses mainly on requirements, architecture details of the SPF. Details of the real-time operating system, board support package, hardware drivers, and high-level software stack are beyond the scope of this document.

1.3 Technical assistance

For assistance or clarification on information in this document, submit a case to Qualcomm Technologies, Inc. (QTI) at https://createpoint.qti.qualcomm.com/.

If you do not have access to the CDMATech Support website, register for access or send email to support.cdmatech@qti.qualcomm.com.

Qualcomm Confidential – May Contain Trade Secrets 2022-03-10 09:19:55 GMT nijiasheng@longcheer.com

2 Overview

2.1 High-level architecture

A client-server model is used where a server provides various functionalities, like realizing audio use cases, per the client requirements.

- The server framework provides the methods to plug in and execute algorithms per the use case requirements.
- The client framework acts as bridge between the high-level software stack (like middleware or application) and server framework.
- The platform (hardware subsystem) and operating system (OS) abstraction help the framework to work on different processors and hardware architectures.

Figure 2-1 illustrates the high-level software architecture used in typical audio embedded systems.

Qualcomm
Confidential - May Contain Trade Secrets
2022-08-10 09:49:55 GMT
nijiasheng@longcheer.com

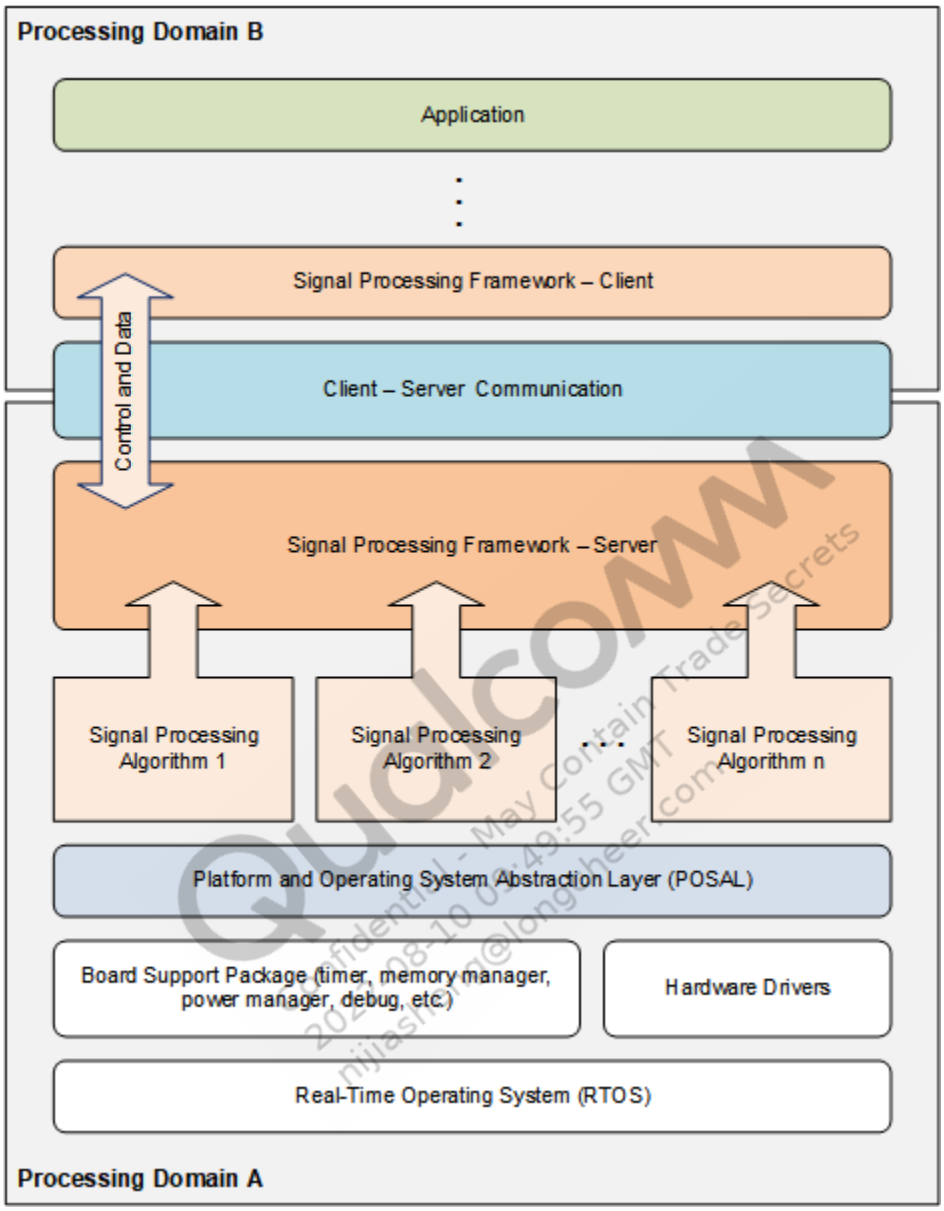


Figure 2-1 High-level audio software architecture

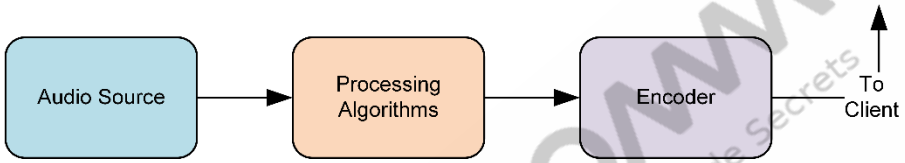
2.2 Possible use cases

The SPF can be used in various audio and voice use cases. This section includes, but is not limited to, the following high-level examples. Detailed use cases are illustrated in Chapter 6.

2.2.1 Audio capture and recording

Audio capture involves the encoding of PCM data coming from a mic or any real-time device (endpoint).

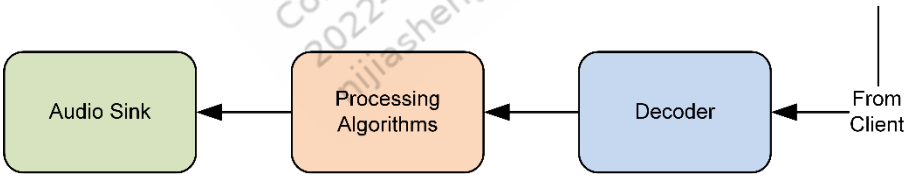
Audio formats used for encoding are: PCM, AAC, WMA std, AMRWB, and so on.



2.2.2 Audio renderer and playback

Audio playback involves the decoding of given audio data, and playing the PCM on a speaker or any real-time device.

Typical audio formats used for decoding are: PCM, MP3, AAC, FLAC, ALAC, AC3, EAC3, Vorbis, WMA std, WMA Pro, DTS, APE, and so on.



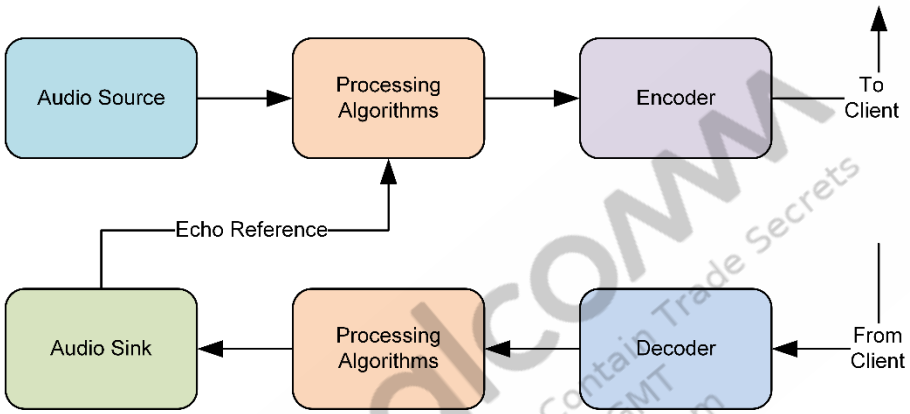
Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 19:49:57 GMT mijiasheng@lonacheer.com

2.2.3 Voice over IP (VoIP)

Voice over Internet Protocol (VoIP) involves both playback and record paths simultaneously, and it is used for voice communication.

Encoder and decoders interact with the host processor application that transmits and receives the data over IP during a conversation with the far-end user.

Typical audio formats used for encoding and decoding are: PCM, AAC, A-law, µ-law, and so on.

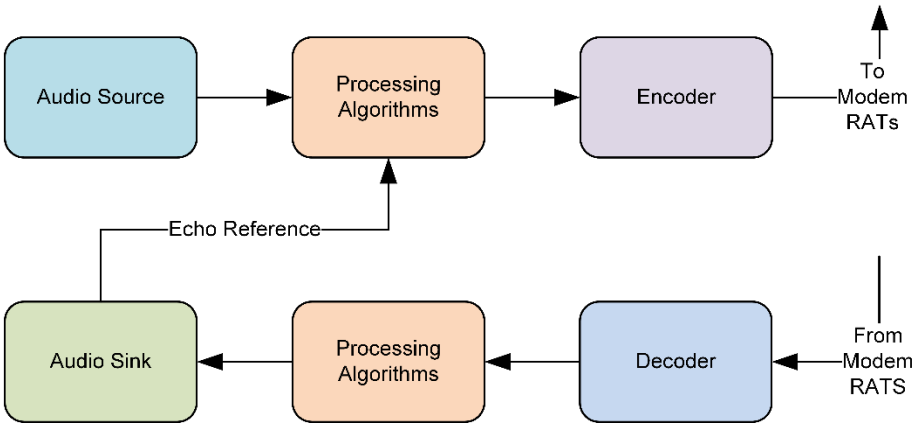


2.2.4 Circuit Switched (CS) voice call

A circuit switched (CS) voice call involves encoding on the uplink path and decoding on the downlink path.

A CS call interacts with modem radio access technology (RAT) layers to transmit and receive the data with the far-end user.

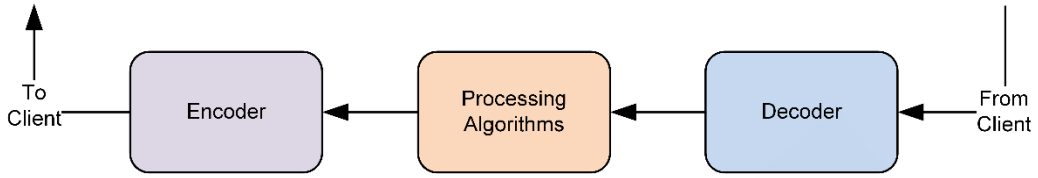
- Example vocoders (encoder and decoder) are: AMRWB, EAMR, EFR, FR, HR, EVRC, EVS, 4GV, G711, G729AB, G722, PCM, V13K, and so on.
- Example RATs: GSM, CDMA, WCDMA, VoLTE



Qualcomm Confidential – May Contain Trade Secrets 2022-08-11 09:20:55 GMT nlyasreng@lcegcheer.com

2.2.5 Audio transcoding

Audio transcoding involves converting one audio format to another. For example, from MP3 to AAC.



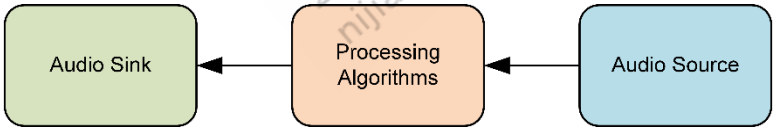
2.2.6 Audio loopback

Audio loopback involves receiving the data from one audio source and rendering it on an audio sink after optional processing.

An audio loopback is used in various scenarios like mixing the side tone in CS voice call, a hands-free profile (HFP), a hearing aid, and so on.

Following are some loopback use cases, where audio must be routed from one device to another device with some conversions.

- PCM to compressed packetized. For example, PCM coming in from a device is encoded as DTS and packetized before transmitting to HDMI.
- Compressed packetized to PCM. For example, data coming from HDMI is depacketized, decoded, and transmitted to a speaker for rendering.
- Compressed packetized to compressed packetized with format conversion.

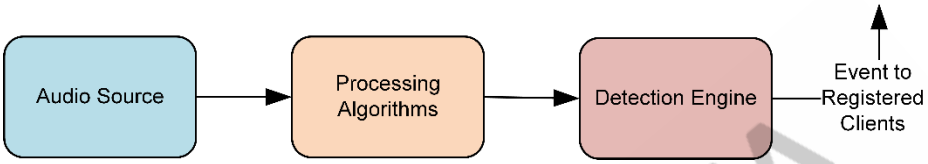


Qualcomm Confidential – May Contain Trade Secrets 2023-08-10 09:49:35 GMT nijiashangelongchen@gmail.com

2.2.7 Audio detection

Audio detection involves receiving the data from an audio source, processing it to improve the signal quality, detecting the intended attribute or event, and informing the registered clients.

Audio detection is used in various scenarios like DTMF detection, keyword detection, audio context detection, and so on.



Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:49:55 GMT nijiasheng@longcheer.com

3 Framework requirements

3.1 General

- Must be processor and platform agnostic.
- Must provide memory scalability options.
- Must provide an option to customize the framework.
- Must provide an option to support various performance modes.
- Must allow use case-specific customizations.
- Must support the unified interfaces to the modules (algorithms and functionality) to interact with the framework.
- Must be scalable to support standalone use cases up to high-end concurrencies.
- Should provide the options to dynamically download the processing modules and algorithms.
- Must support multi-core and multi-Instance configurations to enable distributed audio processing.

3.2 Client interfaces

- Must support client-server communication in the same processor, across the processors, or across the processing domains.
- Must provide the methods to manage shared memory across the client and server framework.
- Must provide generalized interfaces (synchronous and asynchronous) to exchange commands, responses, and events between the client and the server framework.
- Must provide the methods to set up, configure, start, stop, and tear down the use case graphs.
- Must support run-time calibration and monitoring of the modules.
- Must provide the methods to publish the framework and module capabilities, configurations, and calibration interfaces to enable the data driven use case design.
- Should allow proxy clients to handle use case-specific customizations.

Qualcomm
Confidential - May Contain Trade Secrets
2022-08-12 09:49:15 GMT
nijiasheng@longcheer.com

- Should allow modules to preserve the parameters (like converged filter coefficients for adaptive filters) across the use case runs. That is, modules should store the parameters to non-volatile memory while exiting the use case and restore them when the use case is restarted.

3.3 Processing chains, topologies, and graphs

- Must support linear processing graphs (where modules are connected sequentially one by one).
- Should support non-linear processing graphs (where modules are connected as directed acyclic graphs).
- Must support real time (RT) and non-real time (NRT) audio sources and sinks.
- Must support *stream*-based processing graphs where each stream can contain multiple channels.
- Should provide metadata propagation across the processing graphs.
- Should support the processing modules that have different frame-size requirements.
- Must support the option to run multiple instances of the sample module, and the ability to address individual instances of configuration or calibration.
- Must support in-place buffering option for the processing modules in the use case graphs.
- Must provide the methods to support data and control communication between the processing modules.
- Should provide the support for graph-specific functionalities like pause, resume, and flush.
- Should support the modules that take variable input numbers of samples and produce the variable numbers of output samples.
- Should provide the methods to notify the client processor when the last sample of the playback stream is rendered out of the audio sink.

3.4 Media formats

- Must support the fixed and floating point PCM data format.
- Must support various standard compressed data formats and the generic (raw) compressed data format.
- Must provide support for configuring the number of channels, bit width, sample width, and Q factor.
- Must support the media format propagation across the processing modules in the use case.

Qualcomm Confidential – May Contain Trade Secrets 2022-08-10 09:49:55 GMT nijiazheng@longcheer.com

3.5 Scheduling methods

- Should support different scheduling modes and different data delivery mechanisms (buffer availability, timer scheduled trigger, timed packet delivery, deadline driven scheduling, and so on).
- Should provide options to enable custom scheduling and trigger policies to handle complex scenarios.

3.6 Resource management

- Must provide the methods to manage the processing resources (memory management, processor cycles, band width, thread priorities, and so on) required for the use case graphs.
- Should provide the methods to measure the processing requirements of both the framework and modules.
- Should provide the ability to query the delay between required modules.
- Must provide the scalable memory requirement options based on the use cases or capabilities.

3.7 Debugging

- Must provide the methods to log the diagnostic messages.
- Must provide the option to log the audio data (PCM and compressed) at specified locations in the use case graphs.
- Should support different debug levels (which can be featurizable) to debug complex timing issues, memory leaks, and so on.

Qualcomm Confidential - May Contain Trade Secrets 2022-03-16 09:49:55 GMT nijasring@longcheer.com

4 SPF components

4.1 High-level architecture of the SPF

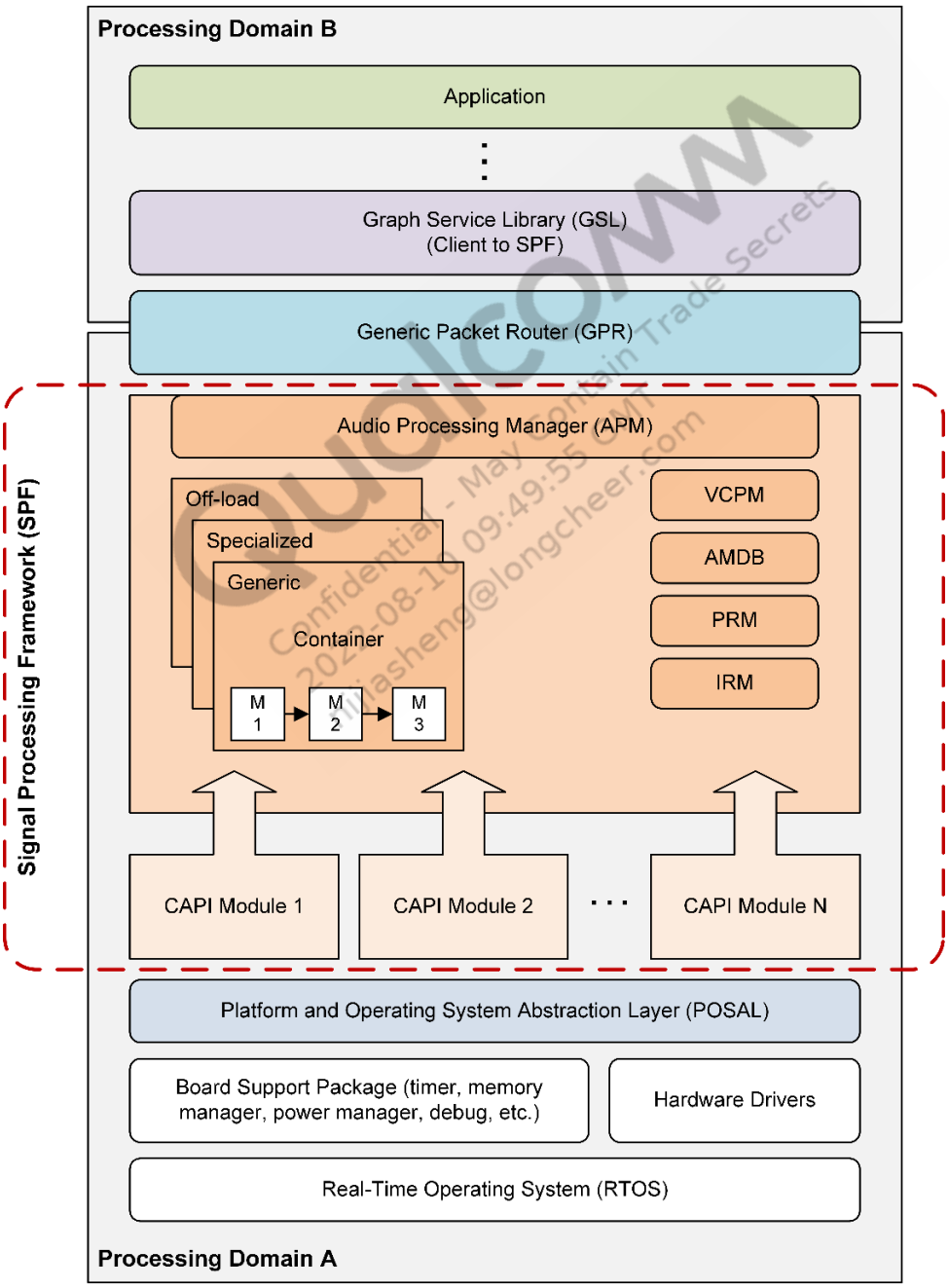


Figure 4-1 SPF high-level architecture

4.2 Functional blocks

This section provides a high-level overview of the functional blocks used in the SPF.

4.2.1 Generic Packet Router (GPR)

The Generic Packet Router (GPR) provides the routing functionality for audio message packets (control, data, events, responses) across the SPF (server framework) and Graph Service Library (GSL; that is, the client framework).

The GPR abstracts the platform-specific interprocessor communication (IPC) transport and protocol layers.

Figure 4-2 represents the GPR header format, which consists of source and destination addresses (domain and port), token (useful for asynchronous communication to match the command and response), and operation code (opcode).

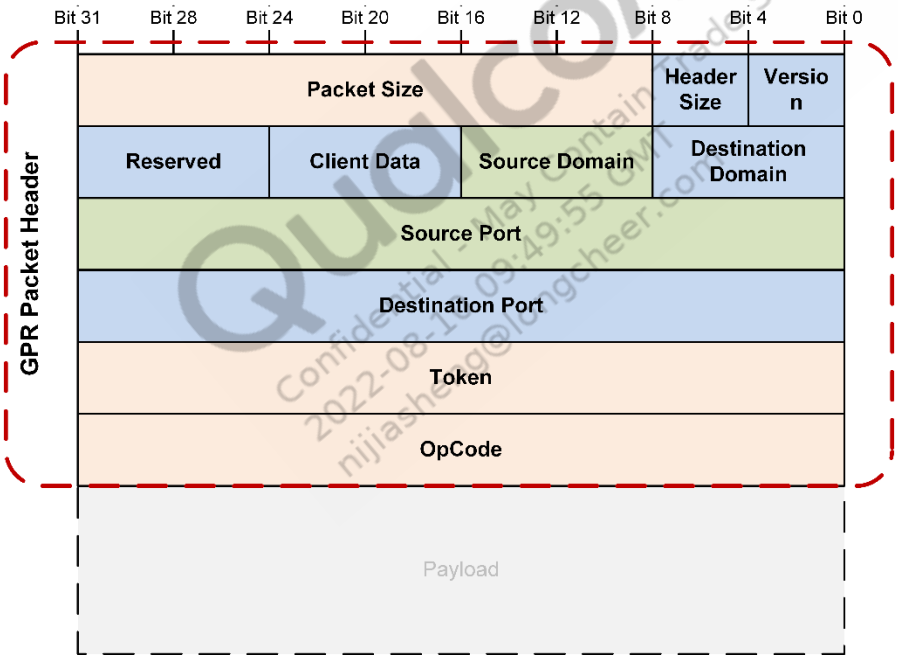
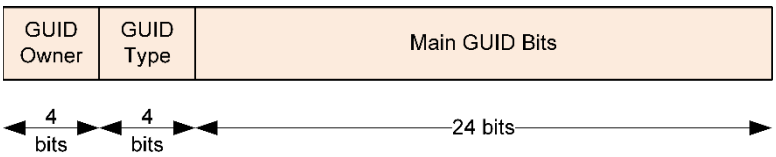


Figure 4-2 GPR header format

4.2.1.1 Opcodes

All opcodes are to follow the GUID format.



- **Owner** – Indicates the owner of the GUID, that is, whether the GUID is defined by the SPF or your custom opcode.
- **Type** – Indicates the specific purpose, for example, control command, control response, data command, data response, event, module identifier, format identifier, CAPI opcode, and so on.
- **Bits** – Used to interpret the message type as an event or response, and to avoid sending the general response or acknowledgment.

4.2.1.2 Messaging between SPF and GSL

The SPF supports two types of messaging approaches for optimal system performance: in-band and out-of-band messages.

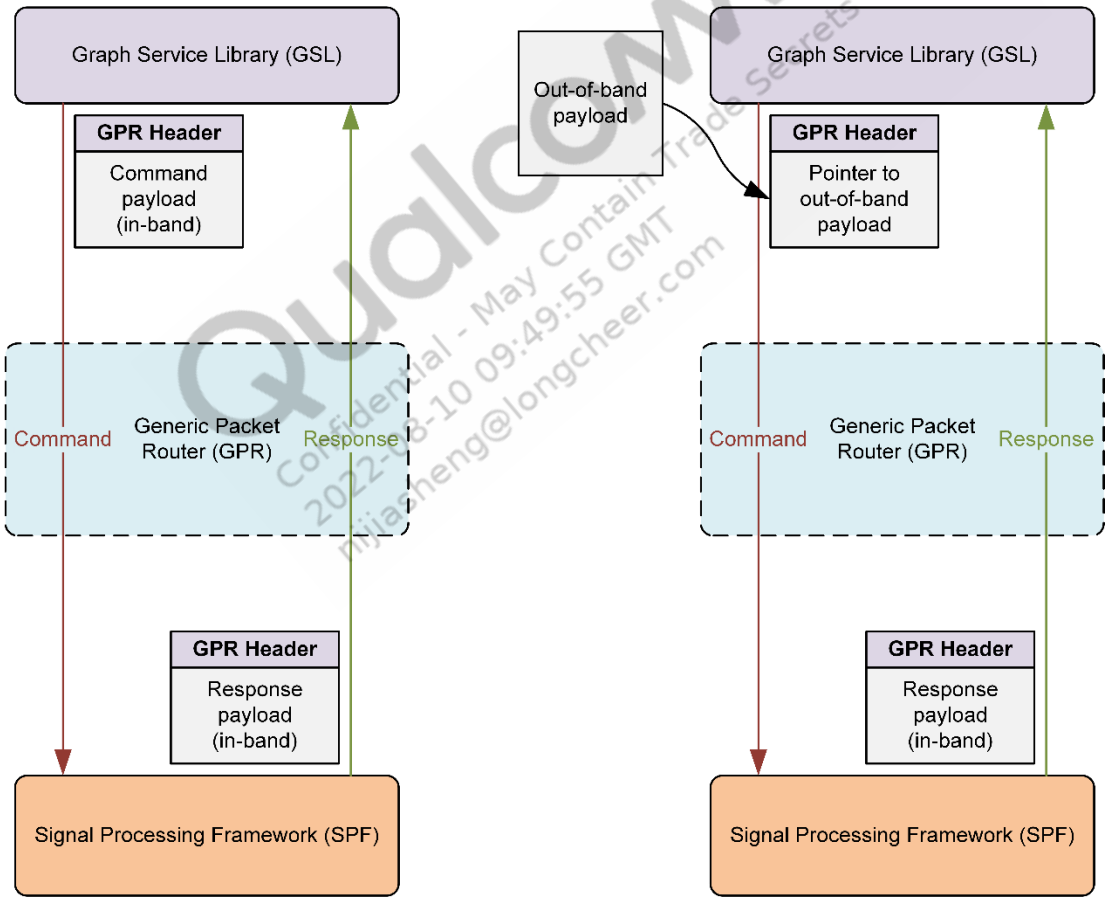


Figure 4-3 In-band and out-of-band messaging methods

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:49:55 GMT mwjasheng@longcheer.com

4.2.1.3 In-band messages

- Contain the actual payload/message as part of the GPR payload.
- The GPR forwards the full packet (that is, GPR header + actual payload) across the process domains.
- Potential memory copies are in intermediate layers between the framework server and client (hence the size limitation to reduce performance impacts).
- Typically used for small payloads, such as <512 bytes (platform-specific configuration), like simple commands, responses, events, and so on.

4.2.1.4 Out of band messages

- Use separate shared memory to keep the actual payload/message.
- The GPR forwards the packet with the address of the actual payload (that is, GPR address + address of the actual payload).
- Allow the framework server and client to access the shared memory without any additional memory copies between them.
- Typically used for larger payloads, such as >512 bytes (platform-specific configuration), like configuration, calibration, data buffers, and so on.

4.2.2 Modules

A module is an addressable *functionality* in the SPF.

- A *module ID* is used to identify the functionality, and it is useful during module instantiation.
- A *module instance ID* is used to identify the instance of the module. It is useful when receiving configuration and calibration information from the clients.
- The module instance (a 32-bit instance ID) should register the callback functions with GPR for receiving messages directly from clients.

Two types of modules are used in the SPF: control modules and data processing modules.

4.2.2.1 Control modules

Control modules provide the public interfaces to the clients (like GML, Codec Driver, and so on) to control the SPF resources and functionalities (each module acts like service that provides specific functionalities).

The Audio Processing Manager (APM), Voice Call Proxy Manager (VCPM), Integrated Resource Monitor (IRM), and Proxy Resource Manager (PRM) are a few examples of the control modules. These modules are not represented in use case graphs.

Qualcomm Confidential - May Contain Trade Secrets 2022-03-10 09:49:57 GMT nijiasheng@longcheer.com

4.2.2.2 Data processing modules

Data processing modules can be static or dynamic, and they are typically wrapped with the Common Audio Processing Interface (CAPI). The CAPI interface acts as the bridge between the framework and core module functionality.

These modules can have zero or more input and output ports. Ports can be control or data, and they are connected to one link at a time (implicit mixing and splitting is not supported at these ports). Modules with zero input and zero output ports are not supported.

Examples of data processing modules include decoders, encoders, postprocessing modules, hardware or software endpoints, DTMF generator (source module), DTMF detection (sink module), echo canceller (multi-port module), and so on.

For more details about the CAPI interface, see the *CAPI for AudioReach Signal Processing Framework API Reference Manual* (80-VN500-6).

4.2.3 Links and connections

Links and connections are used to connect the data processing modules to create the use case graphs or chains. Originating and terminating points of the link are represented by port.

Two types of links are used in the SPF: control links and data links.

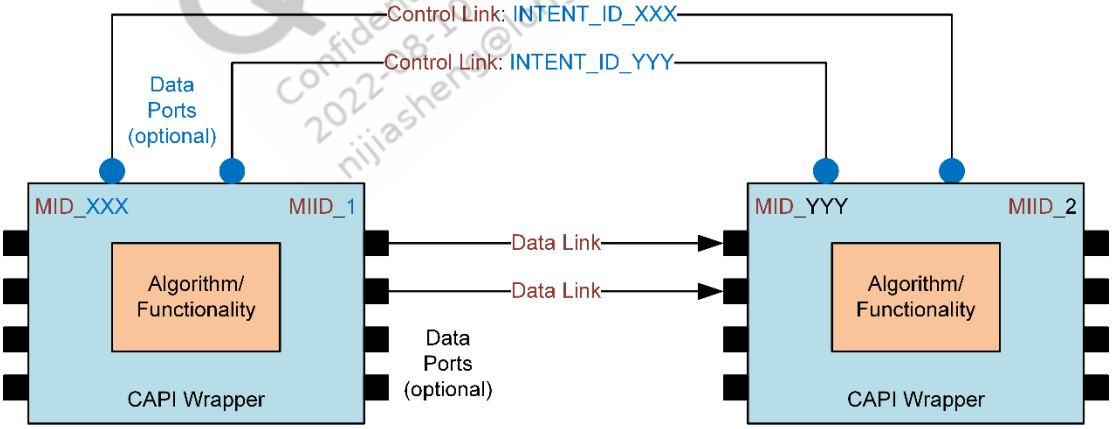


Figure 4-4 CAPI-wrapped modules with control or data links

Qualcomm Confidential – May Contain Trade Secrets 2022-08-16 09:49:55 GMT nijiasheng@longcheer.com

4.2.3.1 Control links

Control links are bi-directional, point-to-point, and dynamic (variable in number) or static (fixed in number with a specific label on the control port).

These links are optional and used in the places where two modules are required to communicate with each other in a steady state without client involvement. They are used for exchanging control messages or intents (that are not required to be synchronous with the data) between the two modules.

4.2.3.2 Data links

Data links are unidirectional, point-to-point, dynamic (variable in number, for example, input to the accumulator module can be variable), or static (fixed in number with a specific label on the data port, for example, an echo reference port can be marked explicitly).

These links are used for exchanging data messages (that need be synchronous with data buffers or samples) between two modules. Each link carries one stream of data that can contain multiple channels.

Data links are not present on the input side for source modules and the output side for sink modules.

4.2.4 Graph and subgraph

A *graph* is the interconnection of a list of data path modules (with input and output ports) to achieve an end-to-end use case.

A *subgraph* is like a graph and is used to represent a section (to control or manage a single unit) of the full graph.

- Subgraph properties provide the necessary configuration for managing a subgraph. For example, performance mode (like low power, low latency, and so on.), use case scenario ID (to handle any use case-specific customizations), and so on.
- SPF clients control the use cases at a subgraph granularity with the help of graph-specific commands like START, STOP, SUSPEND, FLUSH, and so on.

For specific use cases, graph and subgraph mean the same thing.

The SPF supports directed acyclic graphs (no feedback path) only.

4.2.5 Containers

A *container* is a framework implementation that helps in executing a group of data processing modules in the same software thread. Figure 4-5 illustrates this concept.

Qualcomm Confidential - May Contain Trade Secrets 2022-06-10 09:49:55 GMT zhiyishengolongcheer.com

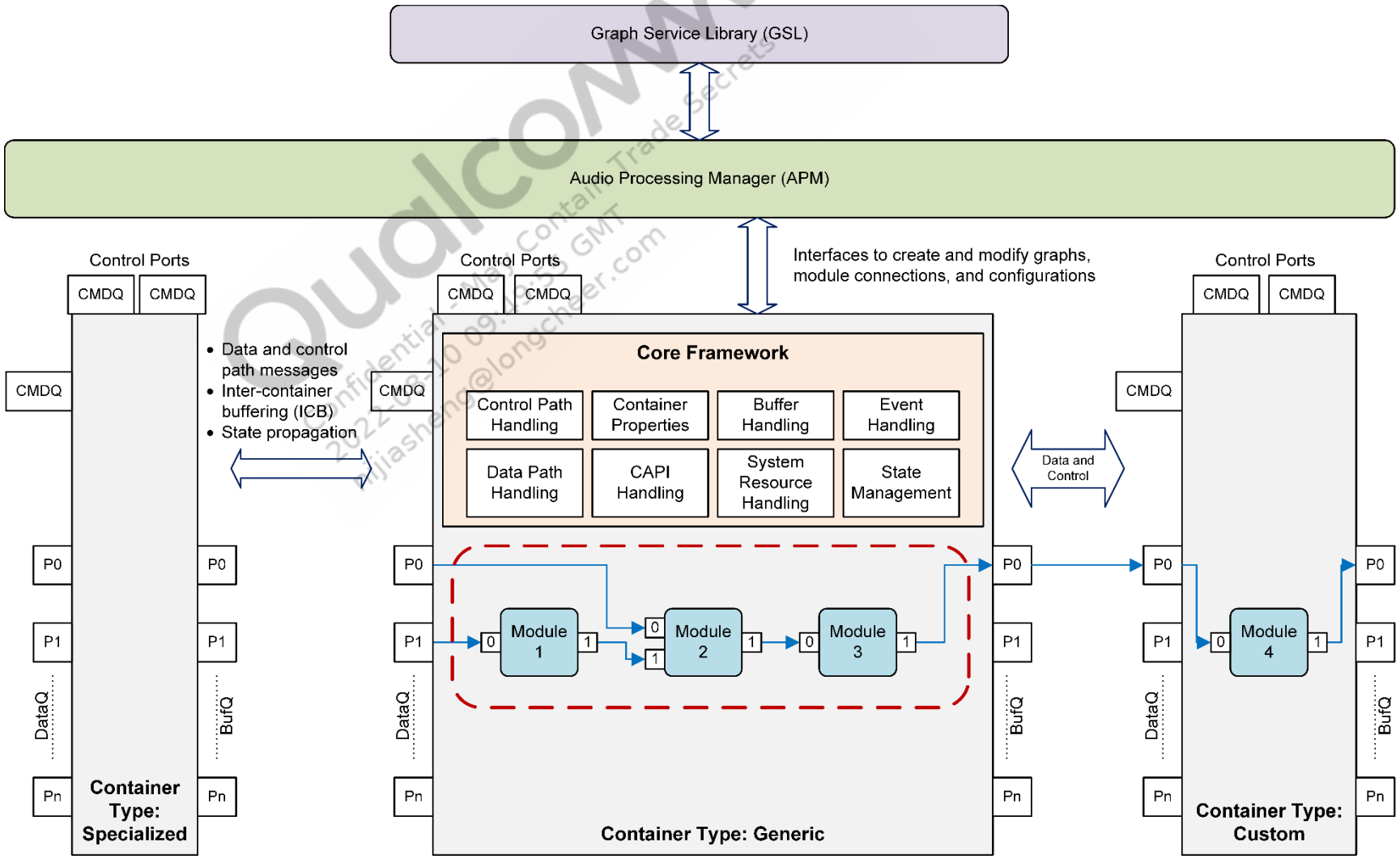


Figure 4-5 Containers

Qualcomm Confidential – May Contain Trade Secrets 2022-03-10 09:10:55 GMT mjashenko@longcheer.com

- Each container instance runs in its own software thread.
- Container properties provide the necessary configuration for container creation and operation. For example, container type, stack size, heap ID, and so on.
- A *Container Type* helps in identifying the specified container and instantiating it during the use case setup (that is, Graph OPEN).
- A *Container Instance ID* is used to represent the instance of the container type, and it is used in the use case graph definition.
- A use case can contain different instances of the same container type to distribute the processing modules in different software threads.

Based on the nature of the framework capabilities required by different use case modules and product needs, there are three container types:

- **Generic container**
  - Supports endpoints (hardware and software), encoders (PCM data to compressed data), decoders (compressed data to PCM data), fixed input and output frame size processing modules.
  - Supports reduced scheduling complexity and memory utilization (which is attractive for low memory applications).
  - Supports variable input and output frame size PCM processing modules, such as fractional resampling.
- **Specialized container**
  - Supports variable input and output frame size PCM processing modules, and different types of synchronization modules used in different use cases.
  - Supports complex scheduling concepts to enable use cases that involve fractional resampling, rate matching, and multi-port modules with different synchronization requirements.
- **Off-load container**
  - Supports reduction of the processing load on the local process domain by helping to off-load an intended module or a group of data processing modules to a different process domain. This is useful in distributed audio processing for effectively utilizing the available hardware resources across the different processors.

The container concept enables framework customization that allows you to create a *custom container* if the reference containers do not support the necessary capabilities with the specified performance. Such customizations can be inter-operable with the reference containers if container-to-container interfaces and behaviors are honored.

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:45:55 GMT jijiaheng@longcheer.com

Containers help with the following:

- Managing (setup, start, stop, teardown) the processing chains (topologies) with CAPI modules.
- Managing the scheduling and trigger policies for the processing graphs inside the container.
- Parallelizing the processing loads in multi-threaded systems.
- Sharing the same stack memory across the modules running in that container.
- Ensuring the appropriate resource requirements (heap, thread priorities, processor and other infrastructure clocks, and so on) required for the modules.
- Managing the data buffering requirements with peer containers through inter-container buffering (ICB) and between the modules in the same container.
- Hosting the necessary command queues to interact with the APM, peer containers, external clients, and internal modules for control messaging.
- Hosting the data queues and buffer queues to exchange the data path messages (data buffers, metadata, end-of-stream (EOS), media format, and so on) with peer containers.
  - These queues are created by the containers when the graph is opened.
  - DataQ is created for every input port of the module that is at the container boundary.
  - BufferQ is created for every output port of the module that is at the container boundary.
- Propagating the port (data and control) state information (start or stop) and nature of the data flow (real time or non-real time) with peer containers through the command queues. This information is useful for updating the scheduling policies at run-time.

Figure 4-6 provides an example dataflow and scheduling policies involved in a simple use case with containers.

Qualcomm Confidential May Contain Trade Secrets 2022-06-10 09:20:55 GMT jinshengc@longchear.com

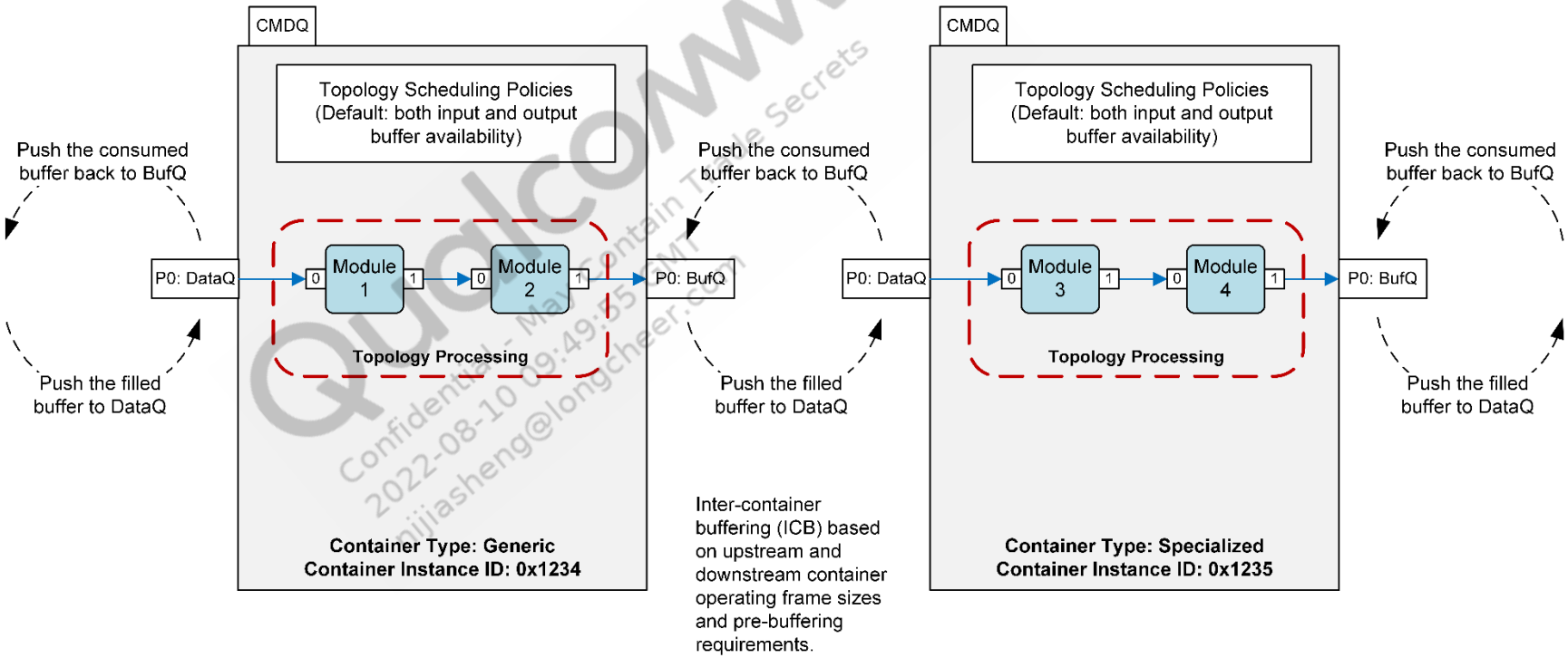


Figure 4-6 Example dataflow across containers

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets
MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

When the use case is started with the default trigger policy:

1. The container triggers or calls its topology processing when both input buffers (filled with data from upstream) and output buffers (free buffers) are available.
2. During the topology processing, input data is propagated through the sequence of CAPI modules (per the use case graph definition) to the output buffer.
   a. After consuming the input buffer completely, the data is pushed back to the upstream' s output queue (BufferQ).
   b. Once the output buffer is filled, the data is delivered to the downstream container input queue (DataQ).
3. These steps are repeated at each container that is driven by buffer availability.

The framework also allows the modules to override the default trigger policies by using trigger policy framework extensions. For example, some modules might want to be called whenever input or output buffer is available (such as the buffering type of modules).

4.2.6 Audio Processing Manager (APM)

The Audio Processing Manager (APM) is responsible for setting up and managing the use case graphs in the SPF.

As illustrated in Figure 4-7, the APM provides generic public interfaces to the GSL client for performing the following graph operations:

- Set up and configure the container processing threads.
- Set up the module's graph within containers.
- Configure and calibrate individual modules within each container.
- Update the run-time graph, including adding and removing containers and modules from the graph.
- Manage data path connections and disconnection across containers.
- Provide run-time calibration of modules within a given container.
- Provide shared memory mapping interfaces.
- Provide path delay between two modules.
- Provide the global framework reset functionality with help of CLOSE_ALL command.

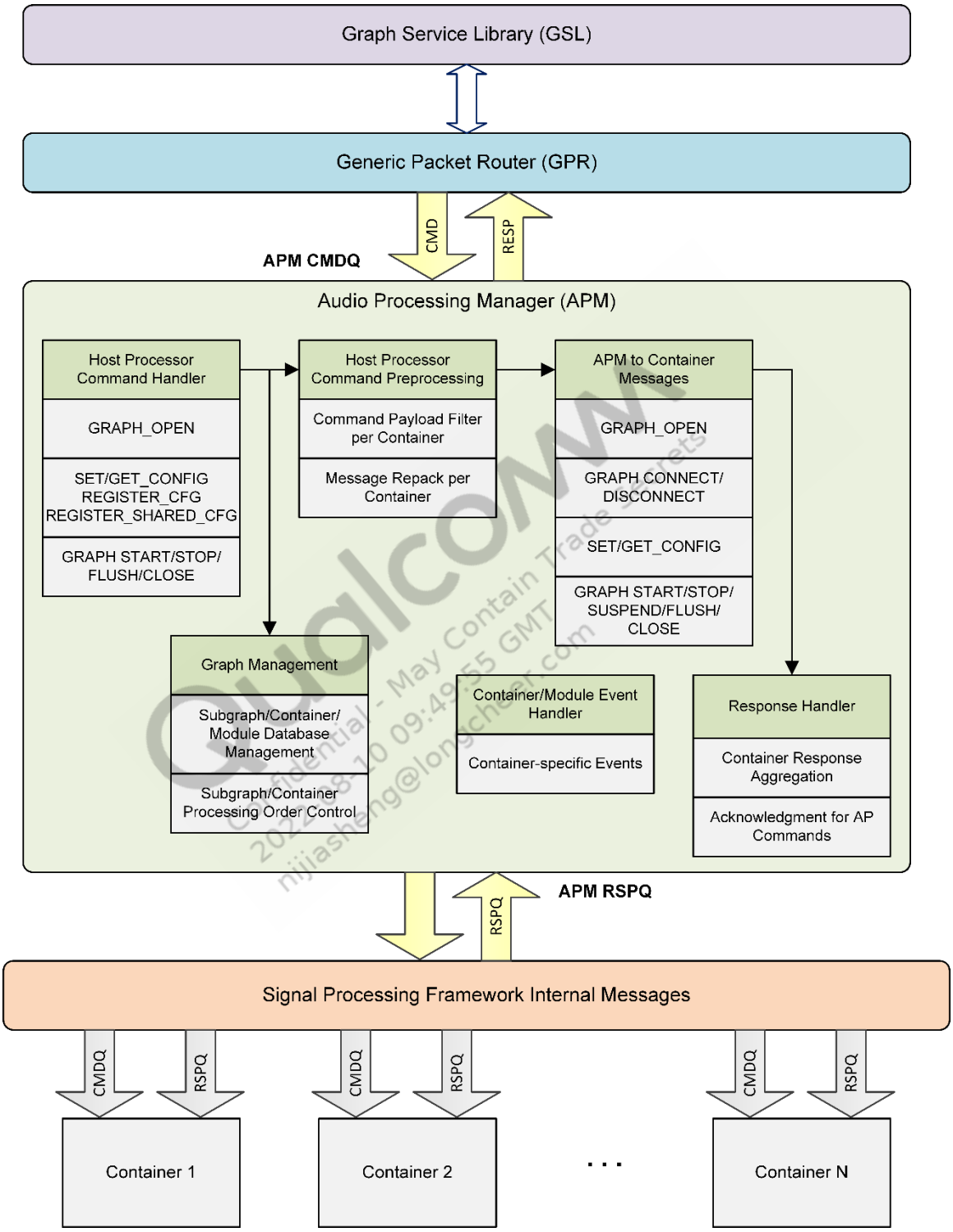


Figure 4-7 Audio Processing Manager

The APM also handles the commands for controlling a subgraph's state machine:

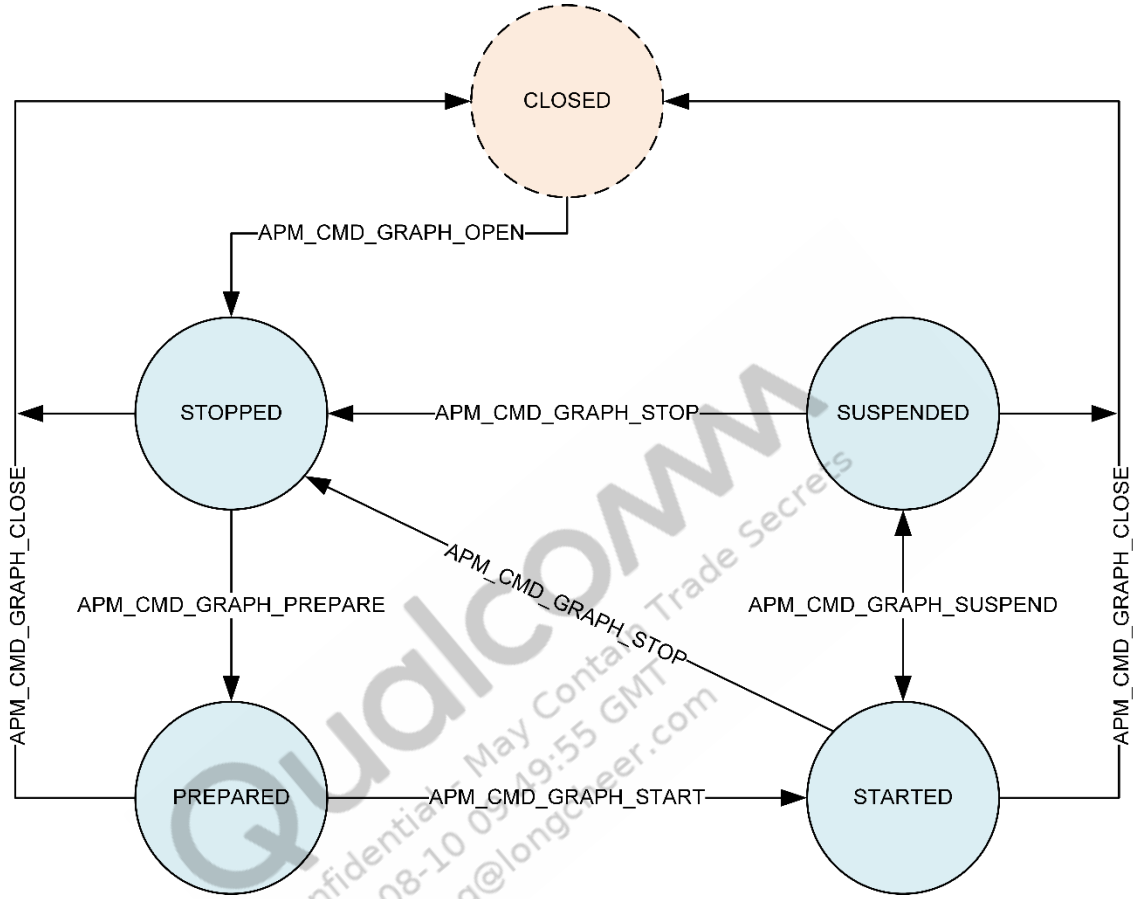


Figure 4-8 APM state machine

- **CLOSED**
  - The logical/non-existent graph state before OPEN and after CLOSE.
- **STOPPED**
  - The state after the graph is opened or transitioned from STARTED as part of STOP.
  - For a START-to-STOP transition, the module algorithmic state is reset, and the container internal data buffers are flushed.
  - Platform-specific resources (MIPS, bus bandwidth, and so on) are de-voted.
- **PREPARED**
  - The media format is propagated through a module topology, if available.
  - The module applies input media format-dependent calibrations.

- **STARTED**
  - Platform-specific resources (MIPS, bus bandwidth, and so on) are voted.
  - When applicable, data triggers, hardware end point, timer interrupts are enabled.
  - A subgraph is ready for handling the data flow.
- **SUSPENDED**
  - Platform-specific resources (MIPS, bus bandwidth, and so on) de-voted.
  - The module algorithmic state and container internal data buffers are maintained and not flushed, unlike the STOP command.

In addition to handling commands from the host processor client, the APM is also responsible for the following operations:

- Depending on the type of command, interact with containers via the framework messaging APIs for sending these commands and handle responses from the containers.
- For a given command, aggregate the container's response for the message and send an aggregated acknowledgement back to the GSL.
- For a given end-to-end graph, manage and coordinate with containers for use cases (such as rate matching) that involve control path data exchange between modules located across different containers.

  This includes utilities for end-to-end container-module graph sorting with respect to data flow direction from the data source to the sink, graph search, and traversal routines.

Qualcomm Confidential – May Contain Trade Secrets 2022-06-10 09:40:35 GMT nijiasheng@longcheer.com

4.2.7 Voice Call Proxy Manager (VCPM)

The Voice Call Proxy Manager (VCPM) acts as *proxy manager* for managing circuit-switch (CS) voice call use cases (due to its functional and performance requirements).

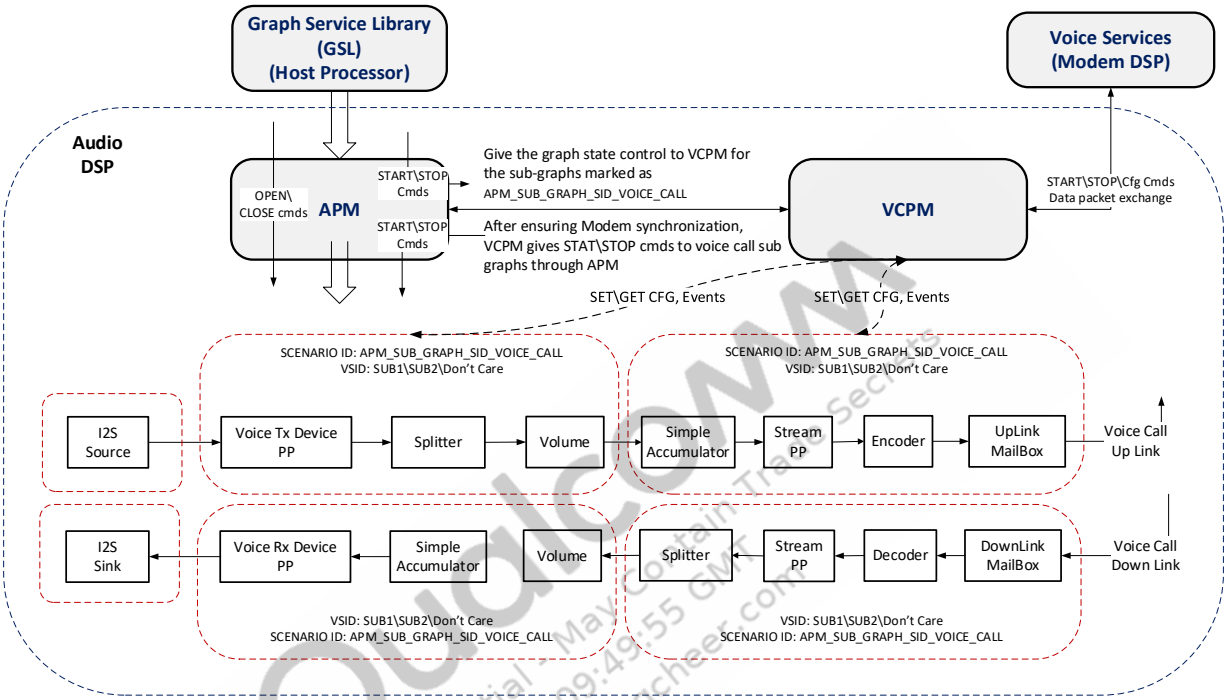


Figure 4-9 Voice Call Proxy Manager

The VCPM is responsible for the following operations.

- Handle the necessary synchronization (dual control) between the application processor client who set up the use case and the modem processor client who acts as bridge between the ADSP and radio access technology (RAT) layers when exchanging the voice packets.
- Handle the inter-network, intra-network handovers and device switches.
- Manage the processing resources for maintaining a specified round-trip delay.
- Handle the multi-dimensional calibration to meet the standard compliance with the algorithm's performance. Following are example keys (optional) to control the calibration dimensions:
  - VCPM_CAL_KEY_ID_NETWORK_ID: GSM, WCDMA, 1-x, and so on
  - VCPM_CAL_KEY_ID_SAMPLING_RATE: 8 kHz, 16 kHz, 32 kHz, 48 kHz ,and so on
  - VCPM_CAL_KEY_ID_VOCODER_CLASS: AMR, EVS, 4GV, EVRC, and so on
  - VCPM_CAL_KEY_ID_WV2: Wide Voice (bandwidth extension) enable

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:49:55 GMT xiaoshangelongcheer.com

  - VCPM_CAL_KEY_ID_VOLUME_LEVEL
  - VCPM_CAL_KEY_ID_VOLUME_BOOST
- Manage dual voice call sessions using subscription IDs like APM_SUB_GRAPH_VSID_SUB1 and APM_SUB_GRAPH_VSID_SUB2.

4.2.8 Audio Module Data Base (AMDB)

The Audio Module database (AMDB) provides the database of CAPI-wrapped modules (both static and dynamic modules) in the SFP.

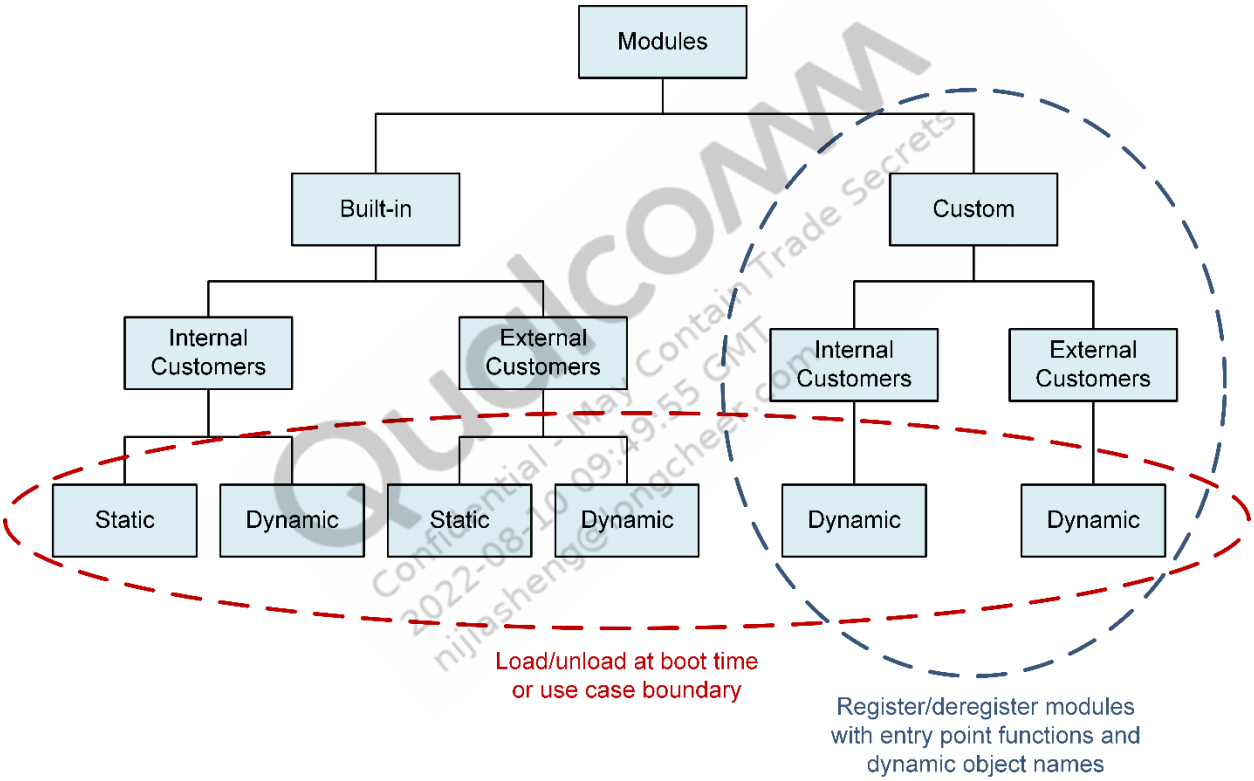


Figure 4-10 Audio Module Data Base

The AMDB is responsible for the following operations.

- Provide the client (to GSL) interfaces to register or deregister custom modules and load or unload the dynamic modules at boot time or use case boundaries.
- Interact with containers for instantiating and tearing down the module.
- Use the platform-specific dynamic download utilities for downloading shared objects to the specified memory (DDR, low-power memory, and so on.).
- Manage the reference counter so that a module will not be unloaded if it is being used by active use cases.

As shown in Figure 4-10:

- **Built-in modules**
  - Modules that are built with the SPF, both static and dynamic objects.
  - The AMDB uses the module database (module ID, entry point functions in autogenerated .c file) that is generated with the build system, so no separate registration or deregistration is required.
- **Custom modules**
  - Modules that are built with the software development kit (SDK) (by both internal and external customers) as dynamic objects.
  - These modules and dynamic objects must be registered with the AudioReach calibration and configuration tool (Qualcomm® QACT™ Platform) using the custom module integration workflow, which in turn registers them with the AMDB in the SPF.
- **Static Modules**
  - Static modules are loaded at boot time along with the SPF.
- **Dynamic Modules**
  - Depending on the memory and latency tradeoffs, dynamic modules can be loaded at boot time or at a use case boundary.

Qualcomm Confidential – May Contain Trade Secrets 2022-08-10 09:49:35 GMT nijiasheng@longcheer.com

4.2.9 Proxy Resource Manager (PRM)

The Proxy Resource Manager (PRM) acts as a proxy between SPF clients (like GSL) and underlying platform drivers to manage the hardware resources (like setting up the clocks, reserving DMA channels, and so on). It provides generic resource request and release interfaces to the client.

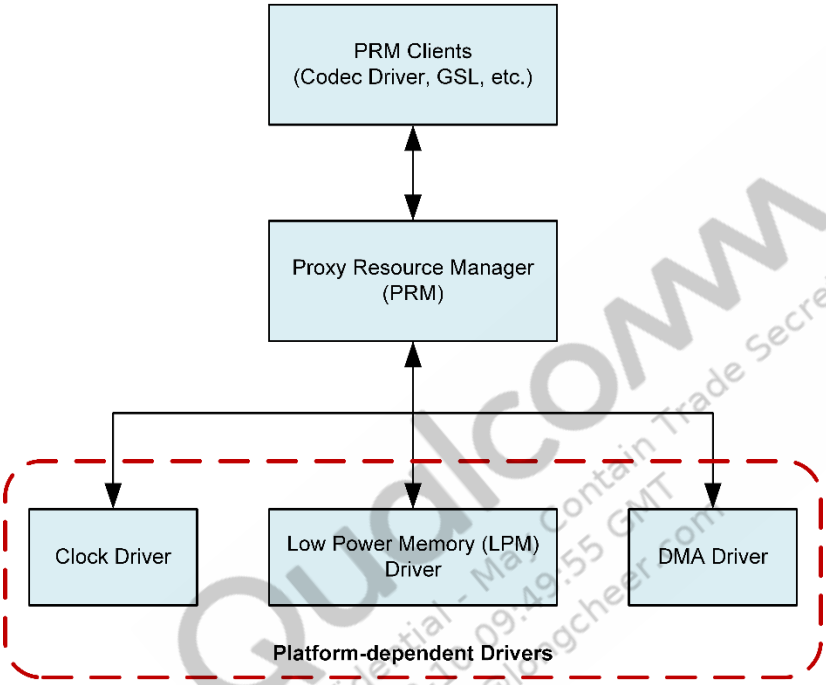


Figure 4-11 Proxy Resource Manager

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:49:55 GMT nijiashen@longcheer.com

4.2.10 Integrated Resource Monitor (IRM)

The Integrated Resource Monitor (IRM) provides the profiling metrics for different resources in the underlying platform.

The QACT platform displays resource metrics (MIPs, bandwidth, heap usage, and so on) for system designers.

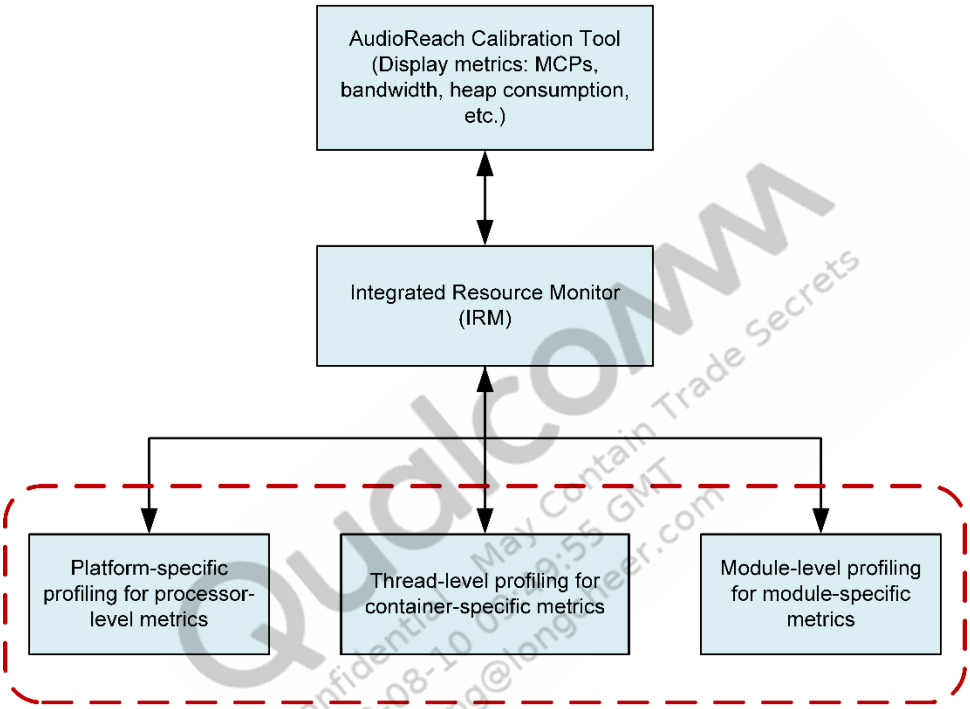


Figure 4-12 Integrated Resource Monitor

Qualcomm Confidential – May Contain Trade Secrets 2022-08-10 03:49:55 GMT nijiasheng@longtheer.com

4.2.11 Platform and OS Abstraction Layer

The platform and operating system abstraction layer (POSAL) provides the necessary abstraction to the framework and makes it processor (hardware) or platform (software) agnostic.

Some of the abstractions include, but are not limited to, the following:

- Power management
  - MIPs/MCPs/MPPs and bandwidth voting to select processor and system bus clocks.
  - Latency voting which could control various performance modes
  - Custom power domains to achieve various power goals
- Processor specific intrinsics to help with optimizations
- Memory Management (different types of heap memories, shared memory mapping and so on)
- Cache memory operations (clean, invalidate operations)
- Conditional and atomic variables
- Priority inheritance and recursive mutex, nmutex
- Signals and Channels (enables to listen multiple signals)
- Interrupts handling
- Timers
- Software threads (priority-based preemptive scheduling)
- Platform specific thread priority data base
- Data (PCM or Compressed) and message logging for diagnostic purposes

Qualcomm Confidential – May Contain Trade Secrets 2022-08-10 09:49:55 GMT yijia.heng@longcheer.com

4.3 Calibration and configuration

Calibration and configuration comprise the control information provided to the module at setup and runtime to control the module's functionality.

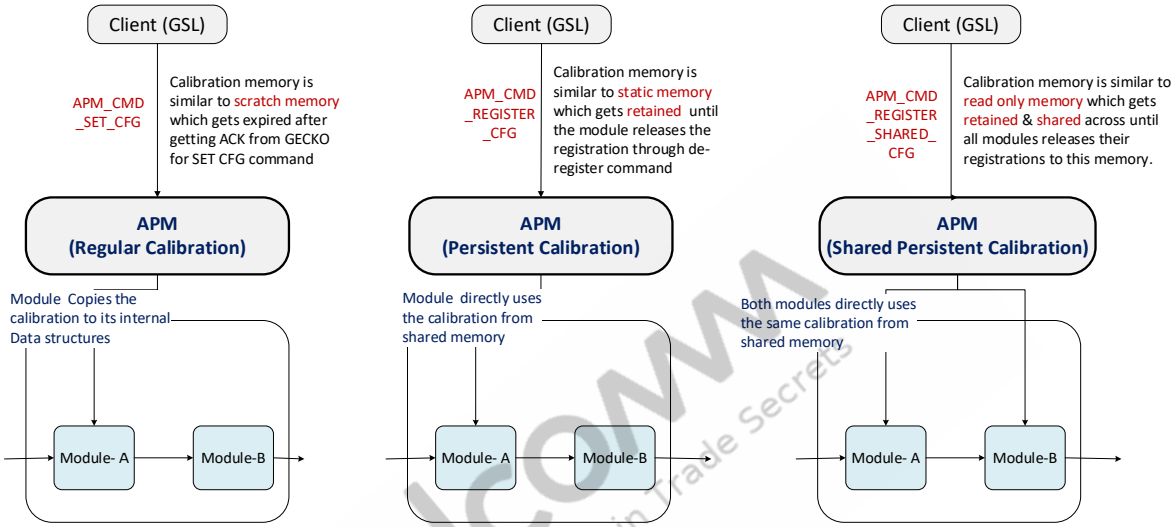


Figure 4-13 Calibration modes

Following are the three types of calibration interfaces exposed by the signal processing framework.

4.3.1 Regular and non-shared calibration

Steps:

1. The client provides the calibration memory to the module.
2. The module copies the calibration data to internal buffers and provides the response to the client.
3. The client frees the shared buffer.

Overheads:

- Additional memory (internal buffer) and buffer copy overhead.
- Hence, we recommend small calibration sizes.

The SPF supports both in-band and out-of-band calibration methods.

Qualcomm Confidential - May Contain Trade Secrets 2022-06-10 09:49:35 GMT hijiashengdongcheer.com

4.3.2 Persistent and shared calibration

Steps:

1. The client provides the calibration memory to the module through an out-of-band payload.
2. The module uses the client-shared memory directly without copying the calibration data to internal buffers, and it provides the response to the client.
3. If the use case is active, the client preserves the shared memory that is being used by the module.
4. The client frees the shared buffer once the use case is closed.

Memory usage and processing cycles are optimal because no copy is involved.

Persistent calibration memory acts like read-write (RW) from the module instance perspective. We recommend this approach for large calibration and configuration blocks (like machine learning models).

Only out-of-band calibration is supported.

4.3.3 Shared-persistent or global-shared calibration

This approach is like persistent calibration. However, in this case, the same data is shared to multiple modules, which can help to save more memory.

Shared persistent calibration memory acts like read-only (RO) from the module's perspective. It is typically used for large calibration or configuration blocks where the same data can be shared across multiple module instances (such as resampler coefficients).

Only out-of-band calibration is supported.

Qualcomm Confidential – May Contain Trade Secrets 2022-08-10 09:18:55 GMT nijiasheng@qualcomm.com

4.4 Multi-instance and multi-core

The SPF enables multi-core and multi-instance system configurations with the help of Master and Satellite architecture. These capabilities help in distributing audio processing loads across the processors to achieve functional and performance goals.

4.4.1 Multi-Instance configuration

Use multi-instance configuration for running disjointed and complete use cases in different process domains.

To rout a specific use case to a specific process domain, the use case designer must provide the hint (through a routing ID). The GSL uses the routing information and interacts with that process domain SPF (also called the *master framework*) to realize the use case.

An example scenario is a system where multiple processors are used for different power profiles:

- Always ON use cases are routed to a low power processor
- Another high-power use case is routed to another processor

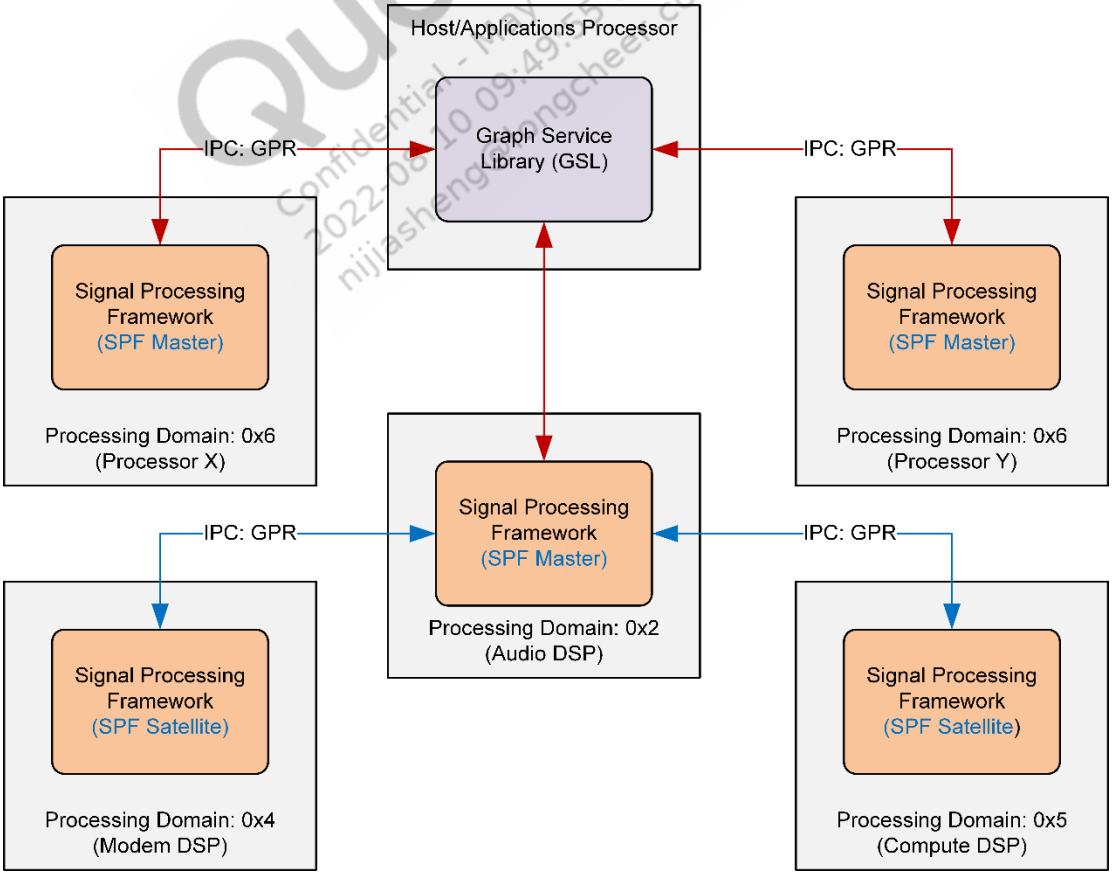


Figure 4-14 SPF in multi-core and multi-instance configuration

4.4.2 Multi-core configuration

Use multi-core configuration to distribute a given use case across multiple process domains. Like a star topology, each off-load graph originates and terminates with the master SPF.

To avoid additional complexities with data or control synchronization and subsystem re-start mechanisms, there is no communication across the satellites.

- The use case designer must provide the hint (by selecting the appropriate process domain) for the modules that are to be off-loaded and distributed.
- The GSL provides this use case information to the SPF (Master), which in turn interacts with the required satellite frameworks to realize the use case.
- Distributed processing helps utilize the processing resources effectively in the system at the cost of potential latency or power increase. Hence the control of selecting the use cases and modules given to the use case designer (with the help of QACT).

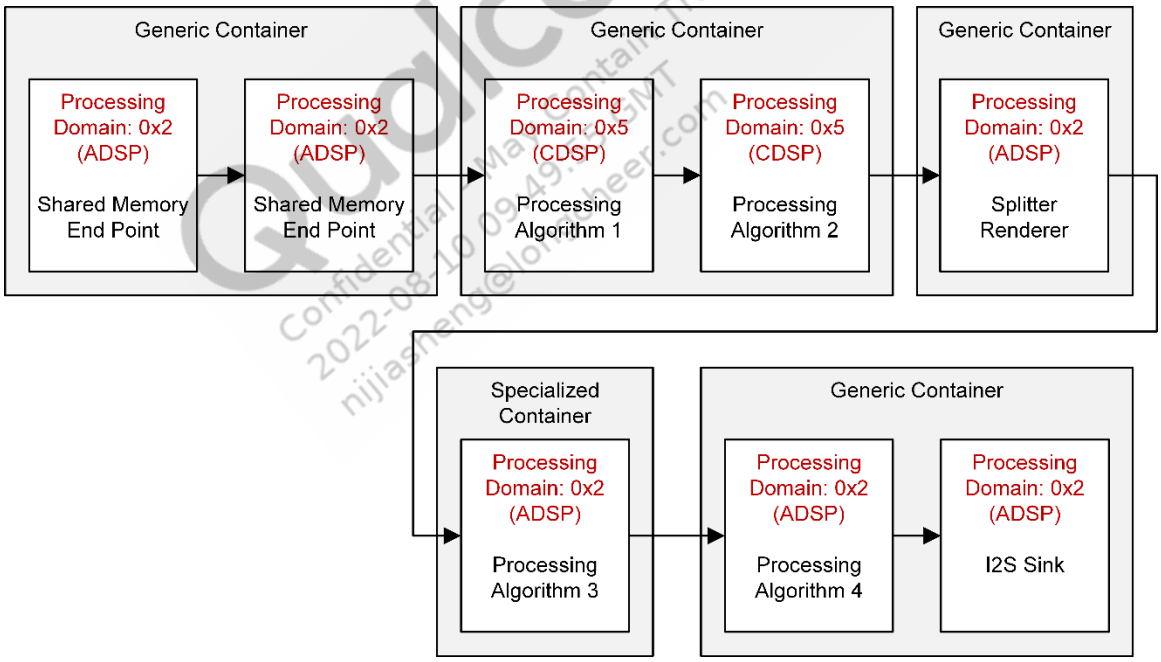


Figure 4-15 Playback with off-load modules – Use case designer's view

Both SPF-Master and SPF-Satellite support most of the capabilities across the process domains, excluding any platform-specific capabilities or modules.

- Figure 4-15 shows the simple playback use case using off-load container to off-load couple of modules into another processor.

- Figure 4-16 is implementation view of Figure 4-15.
  - Off-load container with shared memory end points inserted in the graph to route the data between master and satellite process domains for the given off-load use case.
  - This additional hop across the process domains can increase additional latency due to inter process communication overheads and additional buffering between the off-load container and satellite containers to enable the parallel processing.

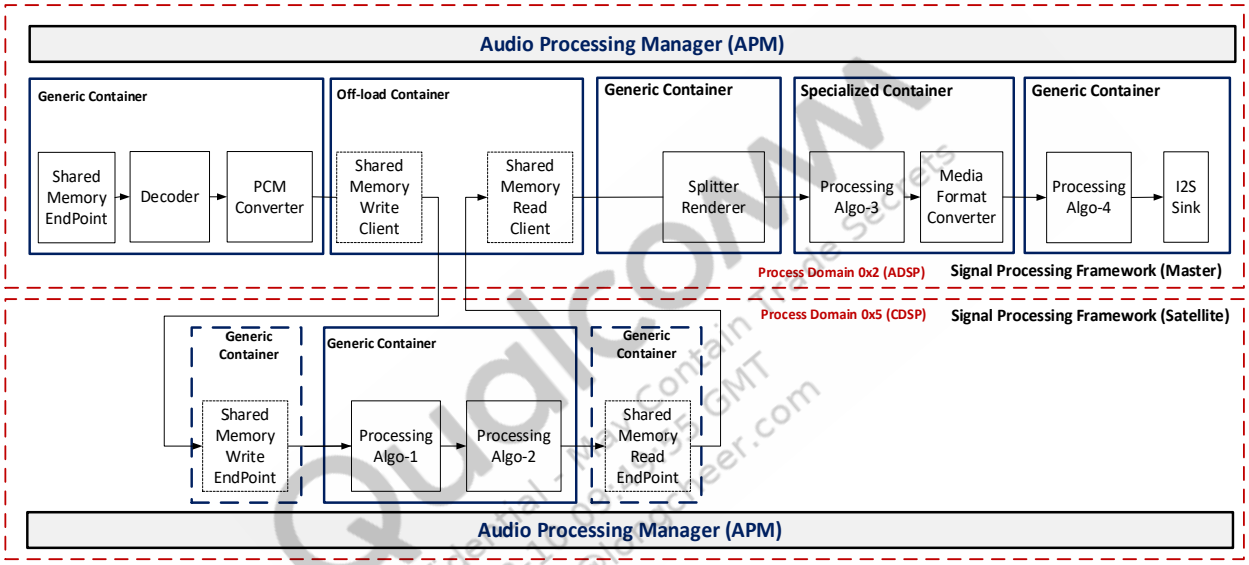


Figure 4-16 Playback with off-load modules – SPF implementation view

The off-load container is inter-operable with other reference containers. It helps to propagate the metadata, end-of-stream (EOS), media format, and state (START, STOP, SUSPEND) information across the off-loading modules.

5 Customizations

5.1 Custom module

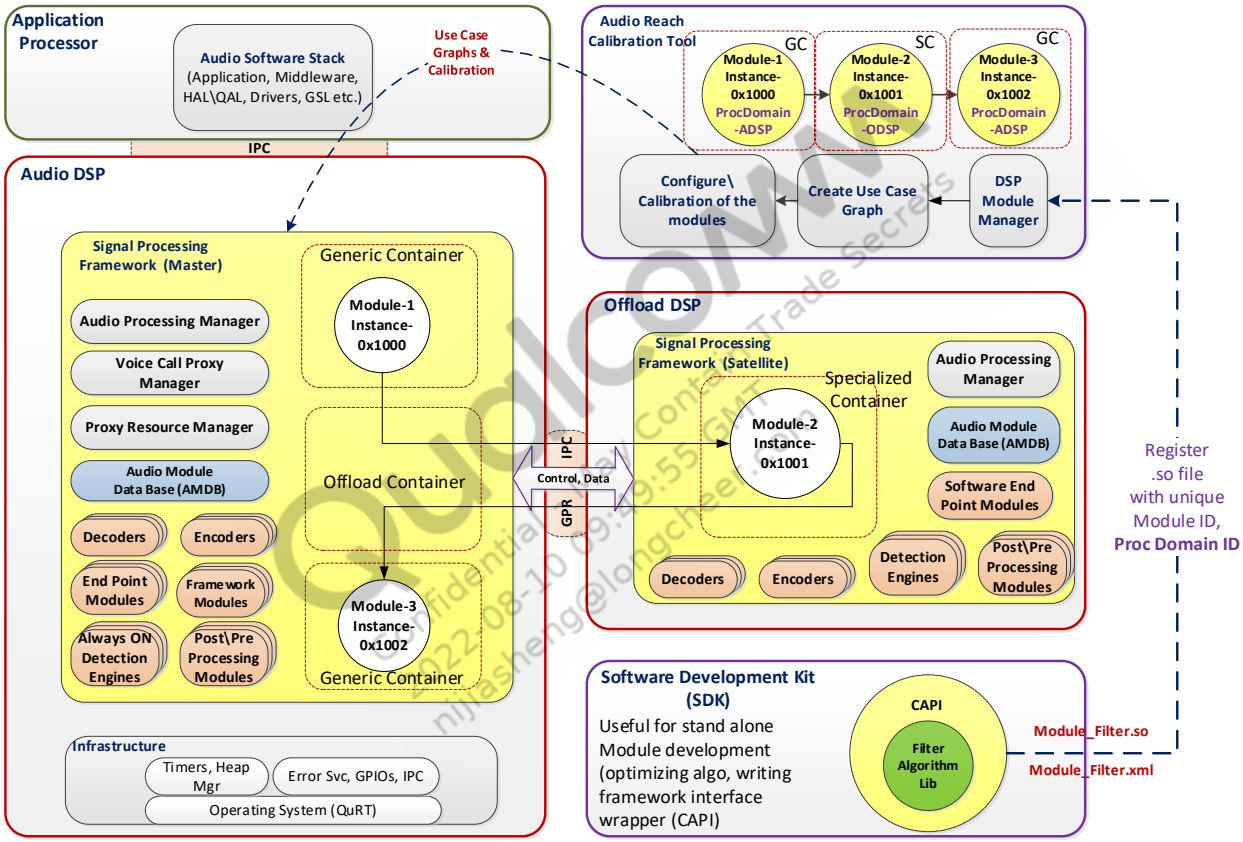


Figure 5-1 Custom module workflow

The custom module development workflow involves the following steps:

1. Use the standalone SDK to optimize a custom algorithm for the specified platform.
2. Develop the Common Audio Processor Interface (CAPI) wrapper for the custom algorithm.

   For examples and detailed instructions, see the *CAPI for AudioReach SPF API Reference Manual* (80-VN500-6).
3. Develop an API header file consisting of Module ID and configuration parameters related to the custom algorithm.

4. Generate an API XML file by running the h2xml conversion tool on the API header file.

   The XML file provides the necessary information about configuration interfaces, supported containers, stack size, and any other policies that are required for the AudioReach configuration tool (QACT platform).

5. Compile the CAPI-wrapped module as a built-in module (a static or dynamic object) by integrating it with the SPF build or standalone shared object in the SDK.
6. Import the custom module into the QACT platform through a module discovery workflow, and create use case graphs by placing the module in the appropriate container and subgraphs.
7. Calibrate or configure the module together with an end-to-end use case, and store the data in the file system (through the ACDB file provided by the QACT platform).
8. Launch the end-to-end use case from the application, which in turn uses the use case graph and calibration information from the ACDB file and provides them to the SPF to realize the use case.

5.2 Custom container

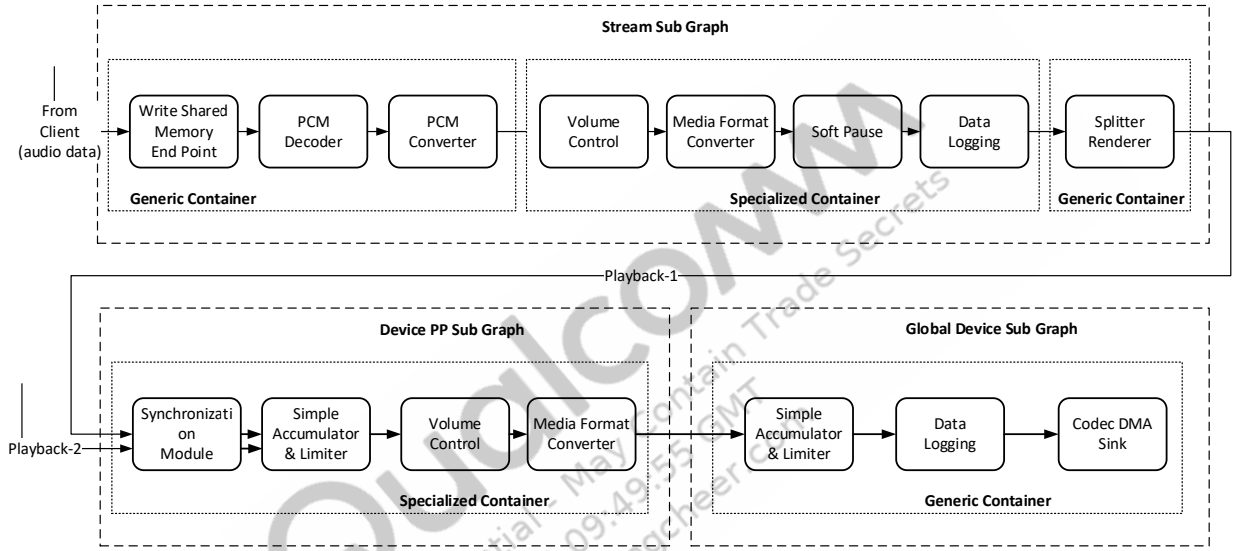
The custom container development workflow involves the following steps:

1. The custom container must use the internal messaging APIs defined between the APM and containers to set up, configure, start, stop, suspend, and tear down the graph.
2. The custom container must use inter-container messaging to make it inter-operable with other reference containers.
3. Develop the custom container per the above guidelines, with specified scheduling policies and performance requirements.
4. Update the supported module headers with this new custom container.
5. Update the internal container data base tables with entry point functions so the APM can create the container during use case setup.
6. Update the APM API header file with the new container type, and regenerate the API XML file (using the h2xml tool).
7. Import the API XML files into the QACT platform to see the new container and supported modules for creating use case graphs.

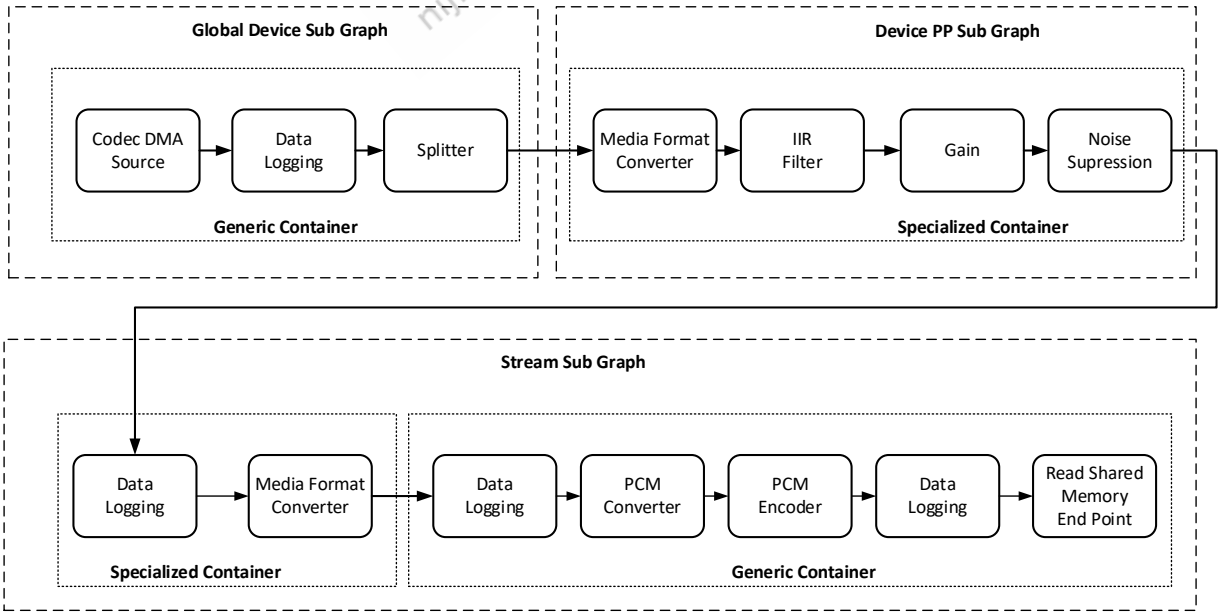
Qualcomm Confidential – May Contain Trade Secrets 2022-08-10 09:19:35 GMT nijchang@longlcmer.com

6 Use cases

6.1 Playback

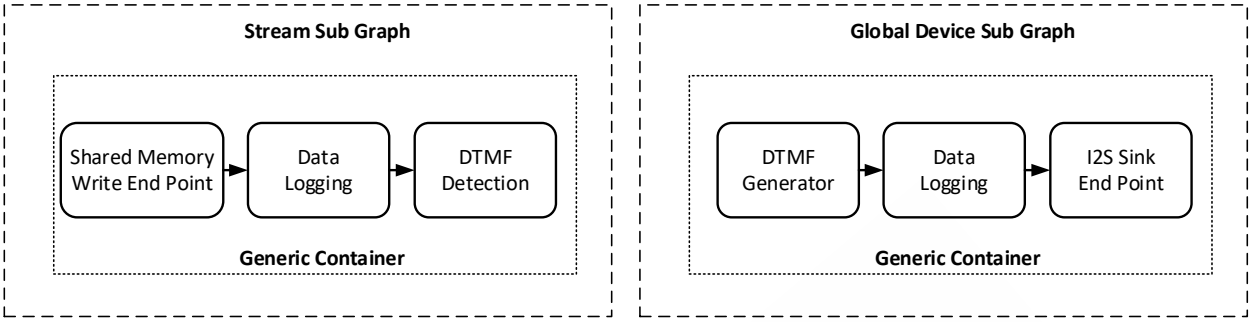


6.2 Capture

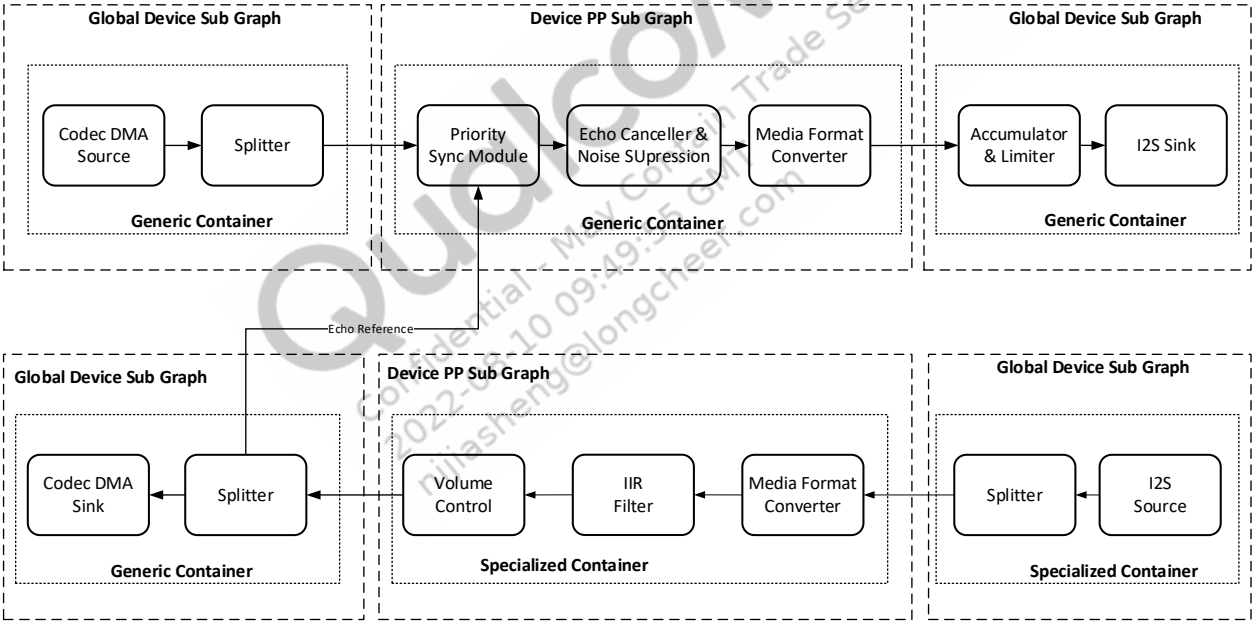


Qualcomm Confidential May Contain Trade Secrets 2022-08-10 09:49:55 GMT nijiasheng@longcheer.com

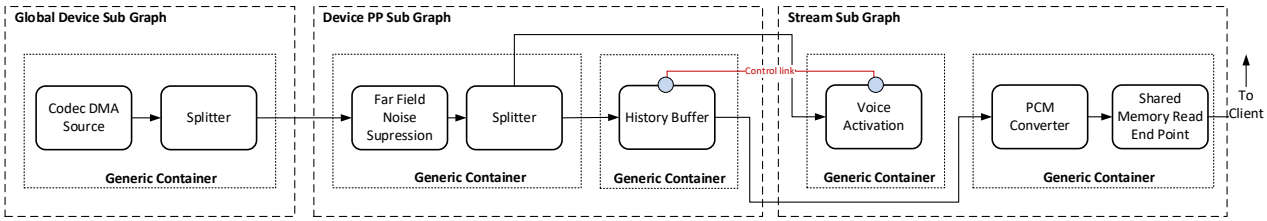
6.3 Use cases with source and sink modules



6.4 Hands Free Profile (HFP)

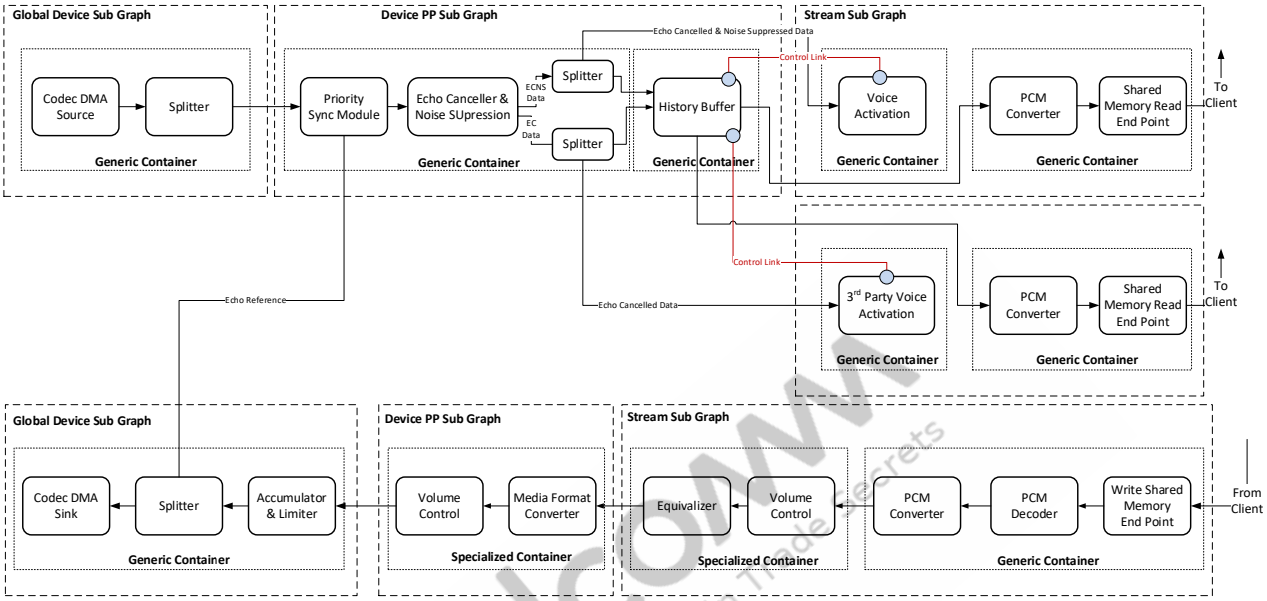


6.5 Voice activation

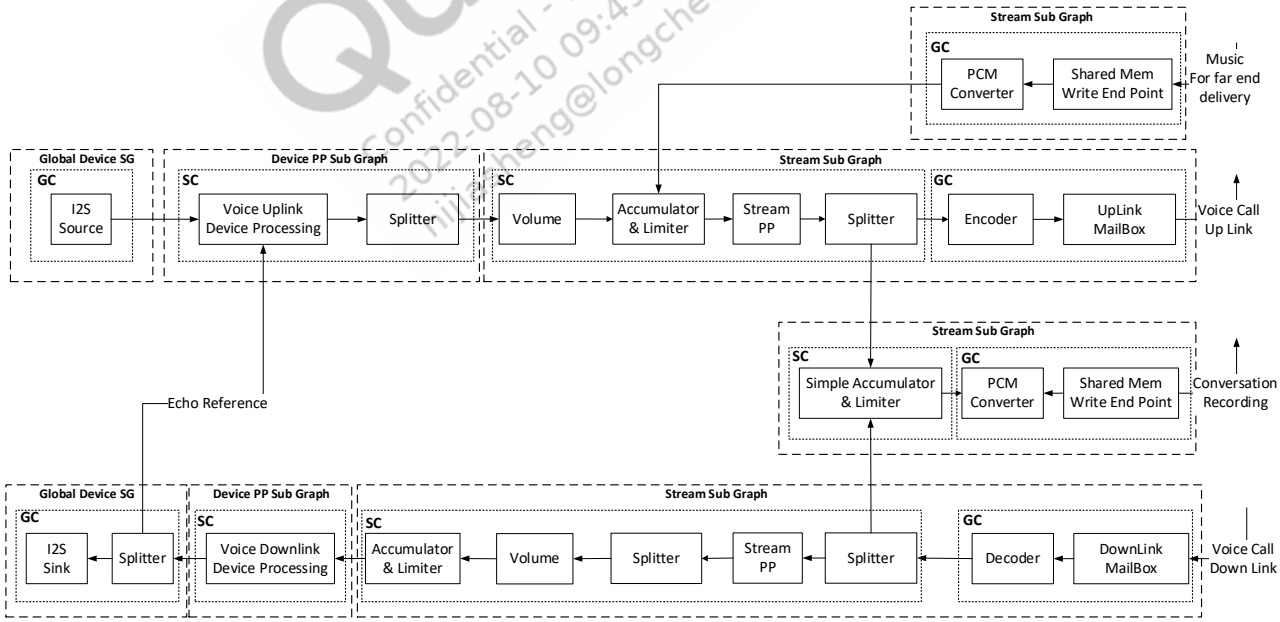


Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:49:35 GMT mijiasheng@longcheer.com

6.6 Voice activation and playback concurrency



6.7 Voice call concurrencies



Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:49:55 GMT hijiazhang@longcheer.com

A References

A.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
CAPI for AudioReach SPF API Reference Manual	80-VN500-6

A.2 Acronyms and terms

Acronym or term	Definition
ACDB	Audio Calibration Data Base
AMDB	Audio Module Data Base
APM	Audio Processing Manager
CAPI	Common Audio Processor Interface
GC	Generic Container
GSL	Graph Service Library
ICB	Inter-container buffering
IPC	Interprocessor communication
IRM	Integrated Resource Monitor
NRT	Non-Real Time
OLC	Off-load Container
Opcode	Operation code
POSAL	Platform and Operating System Abstraction Layer
PRM	Resource Proxy Manager
RAT	Radio access technology
RT	Real Time
RTOS	Real Time Operating System
SC	Specialized Container
SDK	Software development kit
SPF	Signal Processing Framework
VCPM	Voice Call Proxy Manager
VoIP	Voice over IP

Qualcomm Confidential – May Contain Trade Secrets
2025-10-09 09:10:55 GMT
jinsongl@longtheer.com